

Program 6: Deploy an Automation build pipeline using Dockerhub 21/11/2025

Project Structure:

```
prog6
|- Dockerfile
|- server.js
|- package.json
```

Steps for Execution:

Step 1: Move to the directory where we work for DevOps lab. Then create a project directory with the name prog6

```
> mkdir prog6
```

```
> cd prog6/
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~$ cd devops_lab/
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab$ mkdir prog6
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab$ cd prog6/
```

Step 2: Inside the prog6 folder have 2 files in it,

1. Dockerfile 2. server.js

The content which should be included in that file is shown in the below image

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog6$ nano Dockerfile
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog6$ cat Dockerfile
# Use an official lightweight Node image
FROM node:18-alpine

# Create app directory
WORKDIR /usr/src/app

# Copy package.json and package-lock if present
COPY package*.json ./

# Install app dependencies
RUN npm install --production

# Copy app source
COPY . .

# Expose port (same as server listens on)
EXPOSE 3000

# Start command

1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog6$ nano server.js
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog6$ cat server.js
// server.js - simple web server
const http = require('http');

const PORT = process.env.PORT || 3000;

const requestHandler = (req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello from Program6 container!\n');
};

const server = http.createServer(requestHandler);

server.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Step 3: Do the npm initialization by the command,

> npm init -y

This above command adds the package.json into the project folder

Then edit the current package.json in such a way that it should contain the content which is shown in the below image

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog6$ npm init -y
Wrote to /home/chinnu/devops_lab/prog6/package.json:

{
  "name": "prog6",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

Step 4: The final project show contain the files which are mentioned in the below image

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog6$ ls
Dockerfile package.json server.js
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog6$ nano package.json
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog6$ cat package.json
{
  "name": "prog6",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node server.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

Step 5: Now build and tag the image by using the commands,

> docker build -t prog6 .

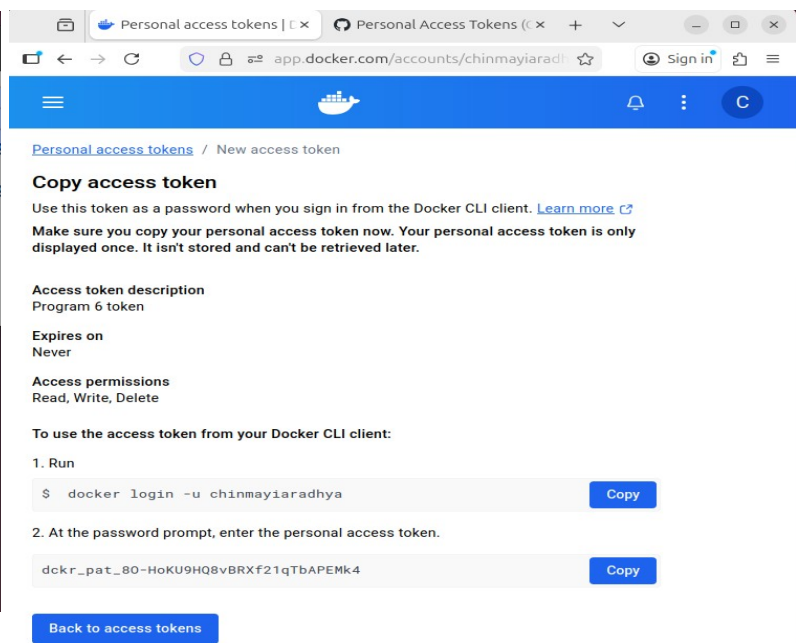
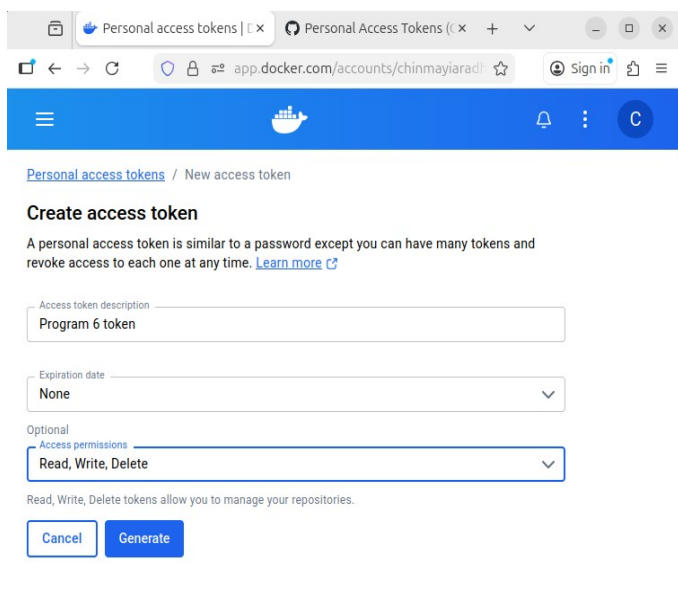
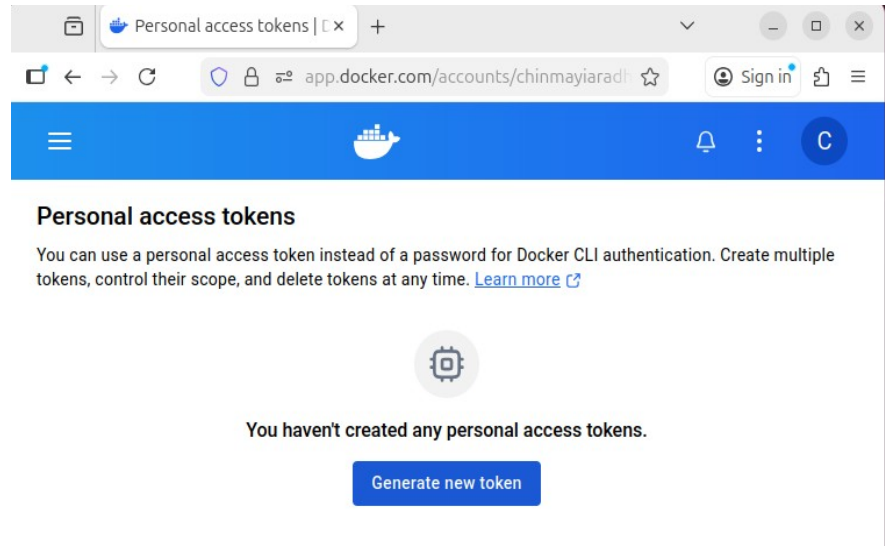
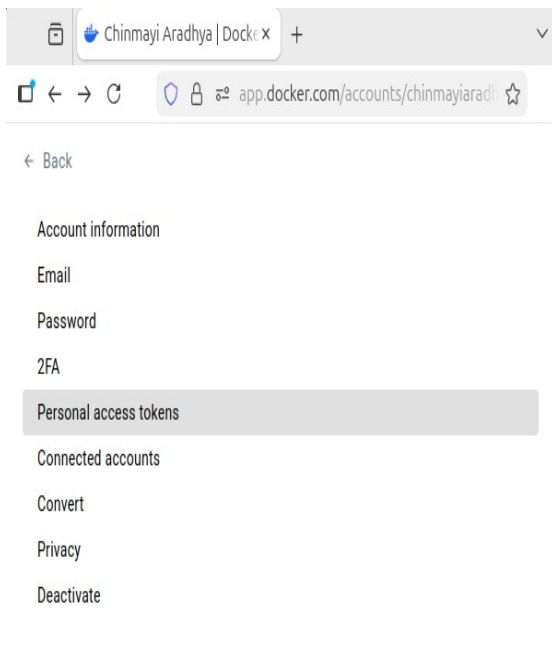
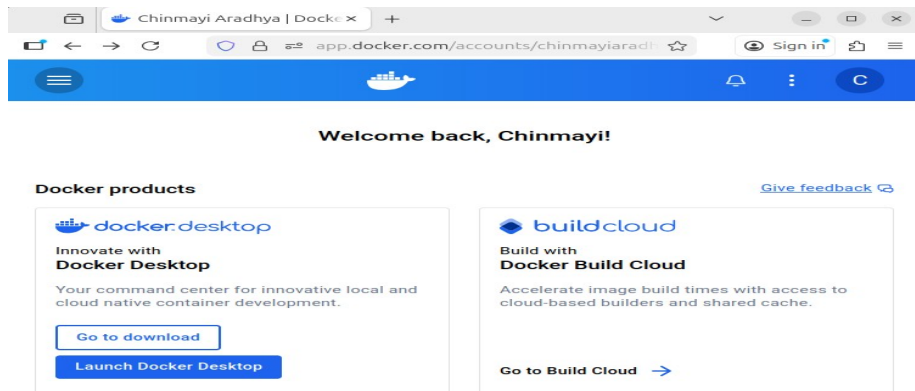
> docker tag prog6 <dh-username>/prog6:latest

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog6$ docker build -t prog6 .
[+] Building 6.1s (11/11) FINISHED
=> [internal] load build definition from Dockerfile                                0.0s
=> => transferring dockerfile: 399B                                              0.0s
=> [internal] load metadata for docker.io/library/node:18-alpine                 3.4s
=> [auth] library/node:pull token for registry-1.docker.io                      0.0s
=> [internal] load .dockerignore                                                  0.0s
=> => transferring context: 2B                                                    0.0s
=> CACHED [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b         0.0s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3        0.0s
=> [internal] load build context                                                  0.0s
=> => transferring context: 1.11kB                                                0.0s
=> [2/5] WORKDIR /usr/src/app                                                    0.0s
=> [3/5] COPY package*.json ./                                                   0.0s
=> [4/5] RUN npm install --production                                           1.9s
=> [5/5] COPY . .                                                                0.1s
=> exporting to image                                                            0.5s
=> => exporting layers                                                            0.2s
=> => exporting manifest sha256:338694a11f3f20291ba3f5661b4f73c017bc0602       0.0s
=> => exporting config sha256:a24ab2c5de9c4f9d5df77be30627615a9e05ab0c16       0.0s
=> => exporting attestation manifest sha256:9f6ca74b3d41bbf27e43b4608742       0.0s
=> => exporting manifest list sha256:53b169be1f8234c35609c62c40dfe8676fc       0.0s
=> => naming to docker.io/library/prog6:latest                                  0.0s
=> => unpacking to docker.io/library/prog6:latest                              0.1s
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog6$ docker tag prog6 chinmayiaradhy/prog6:latest
```

DOCKER HUB

Step 6: Open the browser and search for Docker hub, login with the specific credentials email and password. Then create a Personal Access Token (PAT). Follow the below steps:

> login to docker hub > settings > Personal Access Tokens > Generate new token > fill the description and add the permission of read, write and delete > Generate > Copy the command which will be given to login and also copy the password



Step 7: Come back to the before terminal and login to the docker hub with the login command given in the browser and push the tagged image by using the commands,

> docker login -u <dh-username>

//it will ask for the password, paste the PAT which has been created in the step 5 here

> docker push <dh-username>/prog6:latest

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog6$ docker login -u chinmayiaradhya
```

Info → A Personal Access Token (PAT) can be used instead.
To create a PAT, visit <https://app.docker.com/settings>

Password:

Login Succeeded

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog6$ docker push chinmayiaradhya/prog6:latest
```

The push refers to repository [docker.io/chinmayiaradhya/prog6]

25ff2da83641: Pushed

f18232174bc9: Pushed

8704486a9f75: Pushed

1e5a4c89cee5: Pushed

ccfd53acbeba: Pushed

dd71dde834b5: Pushed

198b87eb3618: Pushed

68a12e8610d6: Pushed

93a6d445d29b: Pushed

Step 8: Verify whether the repository has created by the image in the browser of docker hub,

The screenshot shows the Docker Hub interface for the repository 'chinmayiaradhya/prog6'. The left sidebar contains navigation links: Repositories, Hardened Images, Collaborations, Settings, Default privacy, Notifications, Billing, Usage, Pulls, and Storage. The main content area shows the repository details, including the name 'chinmayiaradhya/prog6', the last push time '3 minutes ago', and a table of tags. The 'latest' tag is highlighted, showing a digest of '338694a11f3f' and an OS/ARCH of 'linux/amd64'.

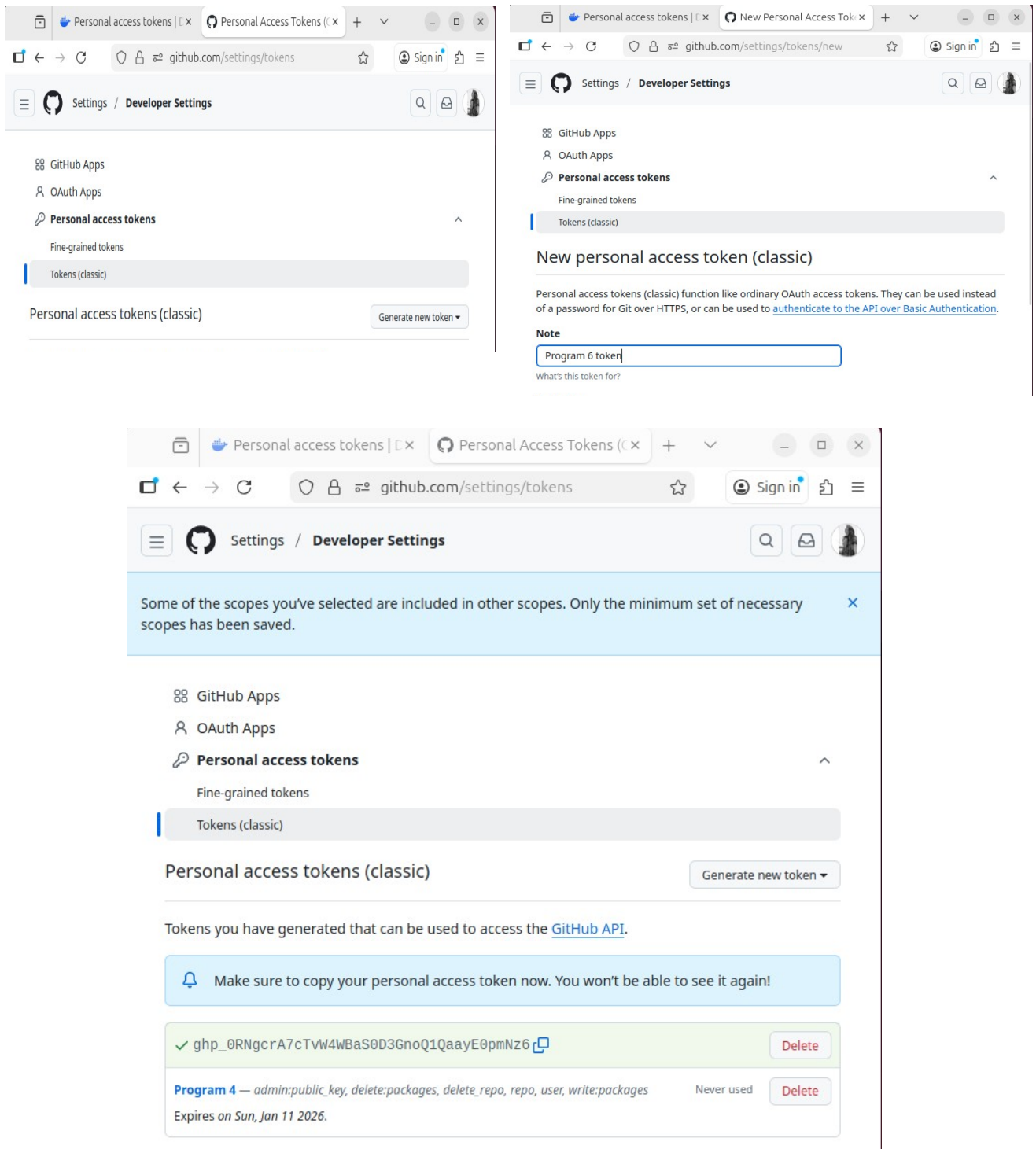
Digest	OS/ARCH	Last pull
338694a11f3f	linux/amd64	less than 1 day

GITHUB

Step 9: Open the browser and search for github, login with the specific credentials email and password. Then create a Personal Access Token (PAT). Follow the below steps:

> login to github > settings > developer settings > Personal Access Token > Tokens (Classic) >

Generate new token > specify the note and select the scope for the PAT > Generate > Copy the PAT which will be displayed after clicking the generate



Step 10: Now login to ghcr by using the command,

```
> echo <PAT-of-github> | docker login ghcr.io -u <gh-username> --password-stdin
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/p
rog6$ echo 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 | docker login ghcr.io -u Chinmayi
-Aradhya --password-stdin
```

WARNING! Your credentials are stored unencrypted in '/home/chinnu/.docker/config.json'.
Configure a credential helper to remove this warning. See
<https://docs.docker.com/go/credential-store/>

Login Succeeded

Step 11: Tag the image which has created in step 4 and push to ghcr by using the commands,

```
> docker tag prog6 ghcr.io/<gh-username>/prog6:latest
```

```
> docker push ghcr.io/<gh-username>prog6:latest
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog6$ docker tag prog6 ghcr.io/chinmayi-aradhya/prog6:latest
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog6$ docker push ghcr.io/chinmayi-aradhya/prog6:latest
The push refers to repository [ghcr.io/chinmayi-aradhya/prog6]
f18232174bc9: Pushed
8704486a9f75: Pushed
25ff2da83641: Pushed
93a6d445d29b: Pushed
dd71dde834b5: Pushed
68a12e8610d6: Pushed
1e5a4c89cee5: Pushed
198b87eb3618: Pushed
ccfd53acbeba: Pushed
latest: digest: sha256:53b169be1f8234c35609c62c40dfe8676fcc8d4f1c8090d11dd2ad2d008f576d size: 856
```

Step 12: Create a repository in github with the name prog6

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1

General

Owner *

Chinmayi-Aradhya

Repository name *

prog6

✔ pro is available.

Great repository names are short and memorable. How about [redesigned-octo-lamp](#)?

Description

Program 6

9 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

Step 13: Now initialize the git and perform the initial commit by using the below commands,

```
> git init
> git add .
```

```

> git commit -m "Initial Commit for Program6"
> git branch -M main
> git remote add origin https://github.com/<gh-username>/prog6.git
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog6$ git init
git add .
git commit -m "Initial Commit for Program6"
git branch -M main
git remote add origin https://github.com/Chinmayi-Aradhya/prog6.git
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/chinnu/devops_lab/prog6/.git/
[master (root-commit) f2541c6] Initial Commit for Program6
3 files changed, 48 insertions(+)
create mode 100644 Dockerfile
create mode 100644 package.json
create mode 100644 server.js

```

Step 14: Now push the folder into the github repository using the command,

```

> git push -u origin main
//here it will ask for the github username and password, for the password paste the PAT which we
have created in step 8

```

```

1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog6$ git push -u origin main
Username for 'https://github.com': Chinmayi-Aradhya
Password for 'https://Chinmayi-Aradhya@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 989 bytes | 989.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Chinmayi-Aradhya/prog6.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

```

Step 15: Open the github browser page again and refresh the repository page, all the files of the folder prog6 will be displayed here

The screenshot shows the GitHub web interface for a repository named 'prog6' owned by 'Chinmayi-Aradhya'. The repository is marked as 'Public'. At the top, there are buttons for 'Pin' and 'Watch' (0). Below this, a navigation bar shows 'main' as the selected branch, '1 Branch', and '0 Tags'. A search bar 'Go to file' and buttons 'Add file' and 'Code' are also present. The main content area displays a list of commits. The first commit is 'Initial Commit for Program6' by 'Chinmayi-Aradhya', with commit hash 'f2541c6', made '5 minutes ago', and containing '1 Commit'. Below this, a table lists the files included in the commit: 'Dockerfile', 'package.json', and 'server.js', each with a file icon and the same commit information.

File	Commit Message	Time
Dockerfile	Initial Commit for Program6	5 minutes ago
package.json	Initial Commit for Program6	5 minutes ago
server.js	Initial Commit for Program6	5 minutes ago